

Day10에 만들어 둔 신청 폼, 오늘은 데이터가 쌓이는 곳을 만든다

Day10 마지막 시간에 "선생님 신청하기" 폼을 각자 만들어 두라는 과제가 나갔다. 지금까지는 제출은 되지만 그 값이 어디에도 남지 않는 테스트용 폼이었다. 오늘은 그 폼에 Supabase 데이터베이스를 연결해, 신청이 들어올 때마다 표의 행 하나로 쌓이게 만든다.

Part 1 - 세팅 (장 7-24)

Supabase 프로젝트 생성부터 배포·환경변수·종단 검증(응답시간 확인 포함)까지, 클릭 하나 단위로 쪼개어 절차다.

Part 2 - Claude Code 실습 (장 25-30)

로그인 확인, CLAUDE.md가 읽히는 장면, 질문 하나 추가 요청, 파일 세 곳이 함께 바뀌는 장면, `/deploy` · `/check` 단축 명령까지 본다.

바뀌는 것 - 신청 폼의 화면과 필드(이름/연락처/지도 과목/지원 동기)는 Day10에 만든 그대로 둔다. 바뀌는 것은 그 값이 어디에 남는가 뿐이다. 폼을 만드는 구조·스타일 감각은 Day8 해피팜 실습에서 가져왔다.

쓰는 법 - 화면 mock은 day9 자료와 같은 부품으로 만들어 실제 창 구조와 같다. 터미널의 실제 실행 원문은 day9에서 그대로 가져왔다.

오늘 끝내야 하는 것 전부 - 번호 순서, 그리고 누가 하는가

[내가 직접] = 계정·클릭·확인·제출. [에이전트+CLAUDE.md] = 코드 수정·커밋·배포 명령·검사. 1-10번은 전부 내가 직접이다 - 아직 에이전트가 등장하기 전이다.

#	오늘 할 일	담당
1	Supabase 가입/로그인 (GitHub 계정 권장)	내가 직접
2	새 프로젝트 생성 - 이름 · 비밀번호 · 리전	내가 직접
3	SQL Editor에 setup.sql 붙여넣기 + Run	내가 직접
4	Table Editor에서 applications 테이블 확인	내가 직접
5	starter 폴더 VS Code로 열기 + npm i	내가 직접
6	1차 배포 (npx vercel --prod)	내가 직접
7	Vercel 환경변수 입력 (SUPABASE_URL · ANON KEY)	내가 직접
8	재배포 (환경변수는 재배포부터 적용)	내가 직접
9	종단 검증 - 폼 제출 + 행 확인 + Logs duration_ms 확인	내가 직접
10	#실습공유에 "A" 중간 보고	내가 직접

왜 전부 직접인가 계정을 만들고, 버튼을 누르고, 화면에 뜬 것을 눈으로 보는 일은 지금은 아무도 대신할 수 없다 - 에이전트는 브라우저를 못 본다. 11번부터 담당이 갈라진다.

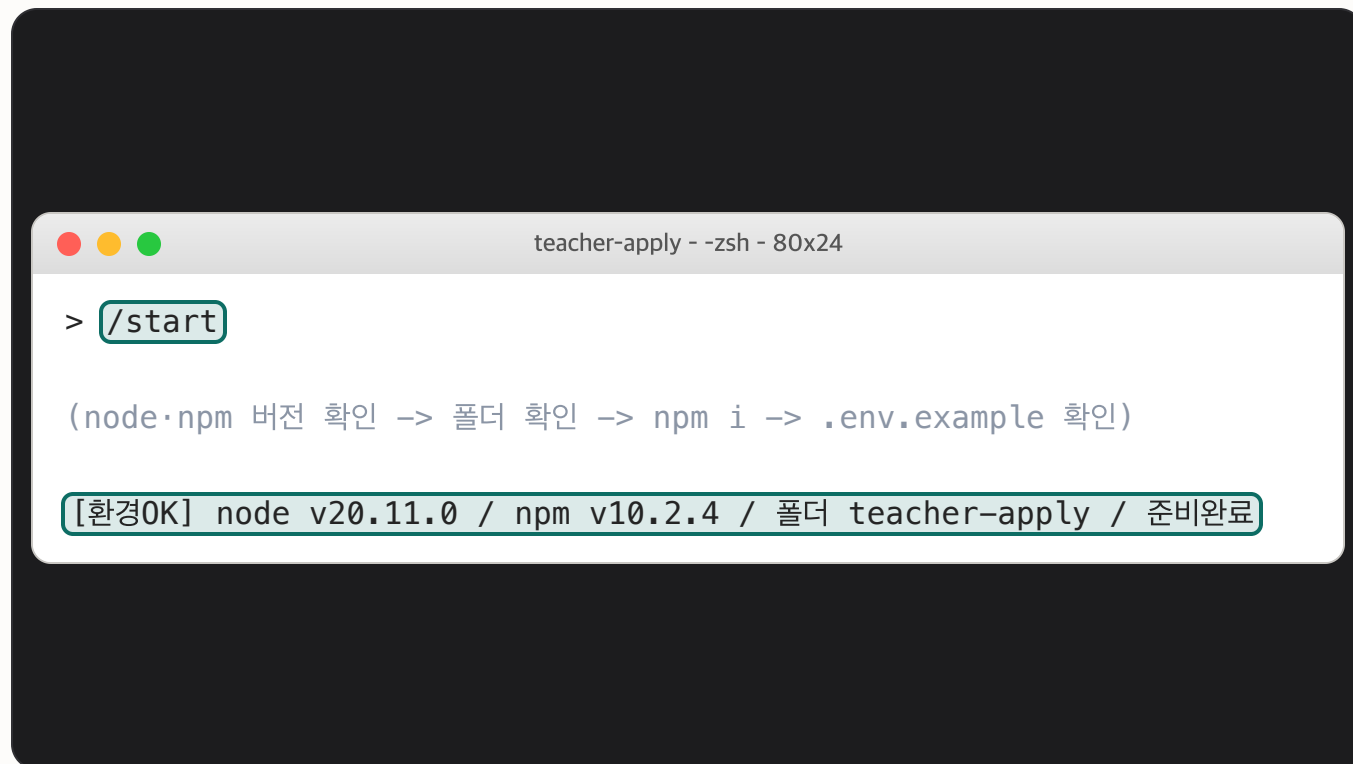
여기부터 Claude Code가 등장한다 - 담당이 반으로 갈라진다

코드 수정 · 커밋 · 배포 명령 · 검사, 이 네 가지가 오늘부터 [에이전트+CLAUDE.md]로 넘어간다. 계정 · 말하기 · 눈으로 확인하기 · 표 안 직접 실행은 여전히 [내가 직접]이다.

#	오늘 할 일	담당
11	Claude Code 로그인 확인 (claude 실행)	내가 직접
12	CLAUDE.md가 읽히는지 확인 (첫 응답에 규칙이 나오는가)	내가 직접
13	첫 지시 말하기 - "질문 하나 추가해줘 (mood)"	내가 직접
14	index.html · api/submit.js · setup.sql 3파일 변경	에이전트+CLAUDE.md
15	setup.sql의 alter문을 Supabase에서 직접 실행	내가 직접
16	/deploy로 커밋 + 재배포	에이전트+CLAUDE.md
17	/check로 중단 재검증 실행	에이전트+CLAUDE.md
18	#실습공유에 최종 A/B/C 보고	내가 직접

바뀌지 않는 것 14 · 16 · 17번도 결과 확인은 결국 내 눈이다 - 에이전트는 파일을 고치고 명령을 실행할 뿐, "성공"이라고 최종 선언하는 것은 오늘도 사람이다(Day9 원칙 그대로).

/start 한 마디가 전원의 시작을 같은 상태로 맞춘다



왜 이 한 줄만 확인하나 컴퓨터마다 node 버전, 폴더 위치, 설치 상태가 조금씩 다르다. 그 차이를 하나하나 눈으로 맞추는 대신, `/start` 가 대신 확인하고 마지막에 영수증 한 줄만 남긴다 - 이 줄이 뜨면 "남들과 같은 출발선에 섰다"는 뜻이다.

이 프로젝트가 미리 정해 둔 것들 폴더 구조 · 모델(haiku) · 키 위치 · RLS · .gitignore · 단축 명령 - 전부 이미 정해져 있다. 지금은 몰라도 실습이 된다. 궁금해지면 `BOILERPLATE-README.md` 를 혼자 읽으면 된다 - 지금은 생각 안 해도 된다.

✓ **성공 표지: [환경OK] 줄이 뜬다**

막히면 [환경막힘] 이 뜨면 그 줄을 그대로 채팅에 올린다 - 원인을 스스로 추측해 다른 걸 고치지 않는다

확인 영수증을 올렸으면 다음 -> **왜 바꾸나 · 장 5**

제출은 되는데 아무 데도 안 남는 폼과, 표에 쌓이는 폼

지금까지의 신청 폼 (Day10 테스트 형태)

- 제출 버튼을 누르면 "제출됐습니다"는 뜨지만, 그 값이 어디로 가는지 확인할 방법이 없다
- 지금까지 몇 명이 신청했는지 셀 방법이 없다
- "코딩 실습 과목을 고른 사람만" 같은 질문에 답할 수 없다
- 다음 사람이 신청 내용을 이어받아 확인할 수 없다

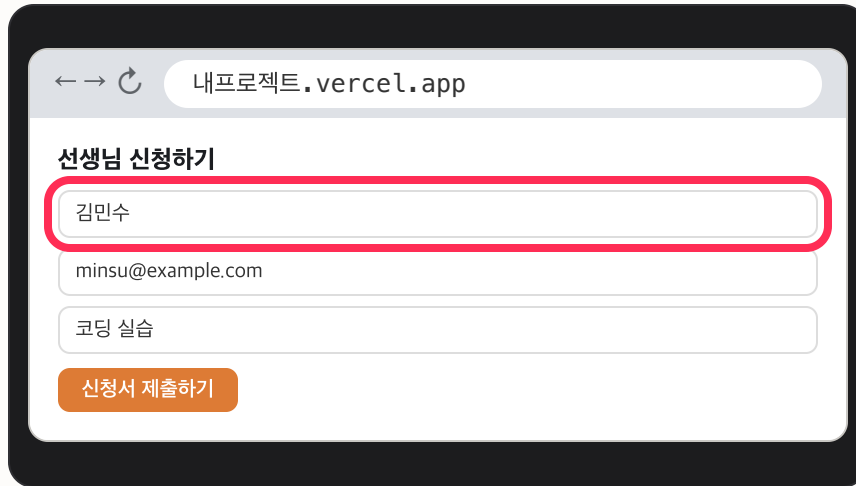
Supabase 표로 받을 때 (오늘)

- 신청 한 건이 **표의 행 하나**로 쌓인다 - 이름/연락처/과목/동기가 각각 칸
- 행 개수 = 신청자 수. 세는 것이 아니라 **바로 보인다**
- "과목이 coding인 행만" 같은 질문에 화면에서 바로 걸러 볼 수 있다
- 같은 표를 **여러 사람이 동시에** Table Editor로 볼 수 있다

오늘 배우는 것 폼 필드(이름/연락처/지도 과목/지원 동기)는 그대로다. 딱 하나, 제출된 값이 도착하는 곳만 "아무 데도 없음"에서 `applications` 라는 표로 바뀐다.

오늘 끝나면 이 두 화면이 이렇게 연결된다

지원자가 보는 화면 - 배포된 신청 폼



강사가 보는 화면 - Supabase 표

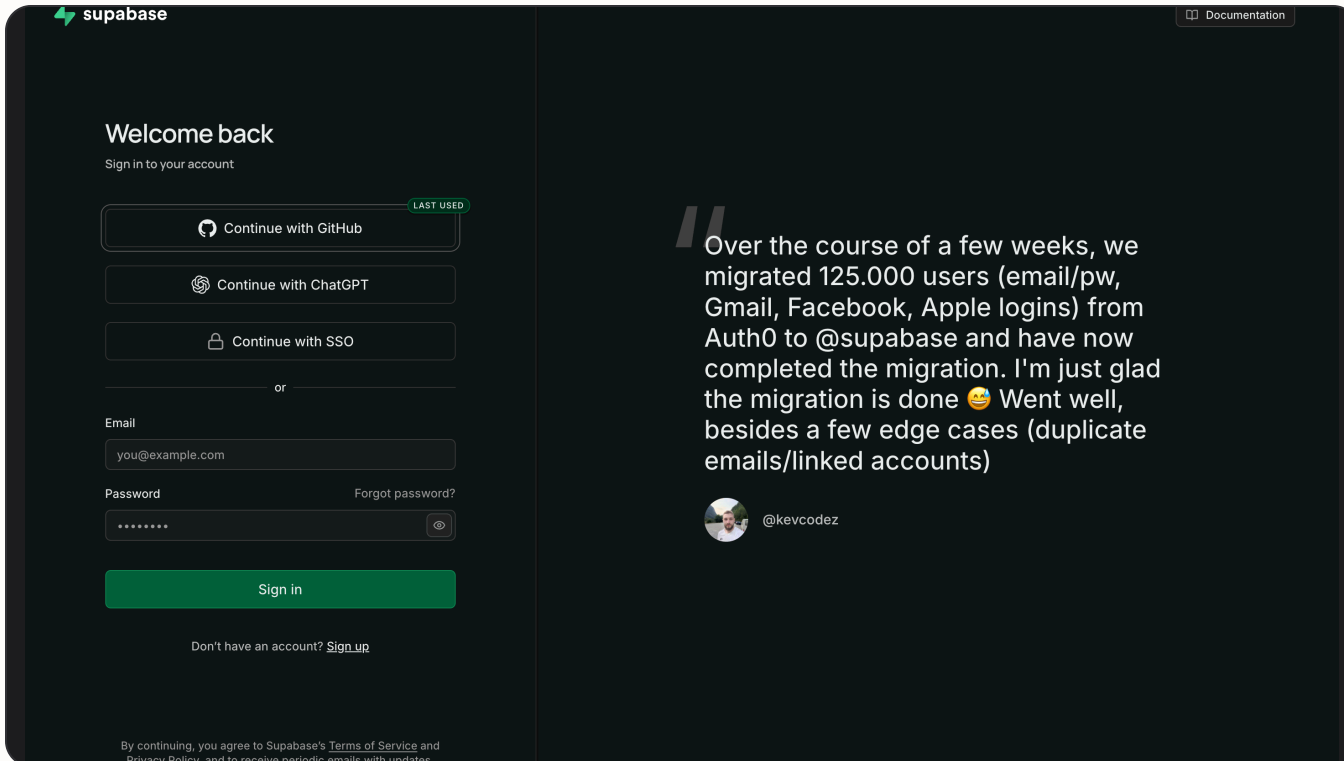
teacher-apply		
name	contact	subject
김민수	minsu@example.com	coding
이서연	010-1234-5678	basic

오늘의 목표 왼쪽 폼에 입력해 제출하면, 몇 초 안에 오른쪽 표에 새 행 하나가 생긴다. 이 연결 하나를 만드는 것이 오늘 전체 목표다.

✓ **오늘의 성공 표지:** 내가 제출한 행이 내 눈에 Table Editor에서 보인다

터미널의 "배포 완료" 메시지는 오늘도 통과 기준이 아니다 - Day9에서 확인한 그 원칙이 오늘도 그대로 적용된다. 다음 -> **S1.1 · 장 7**

브라우저에서 supabase.com을 열고 "Continue with GitHub"를 클릭한다



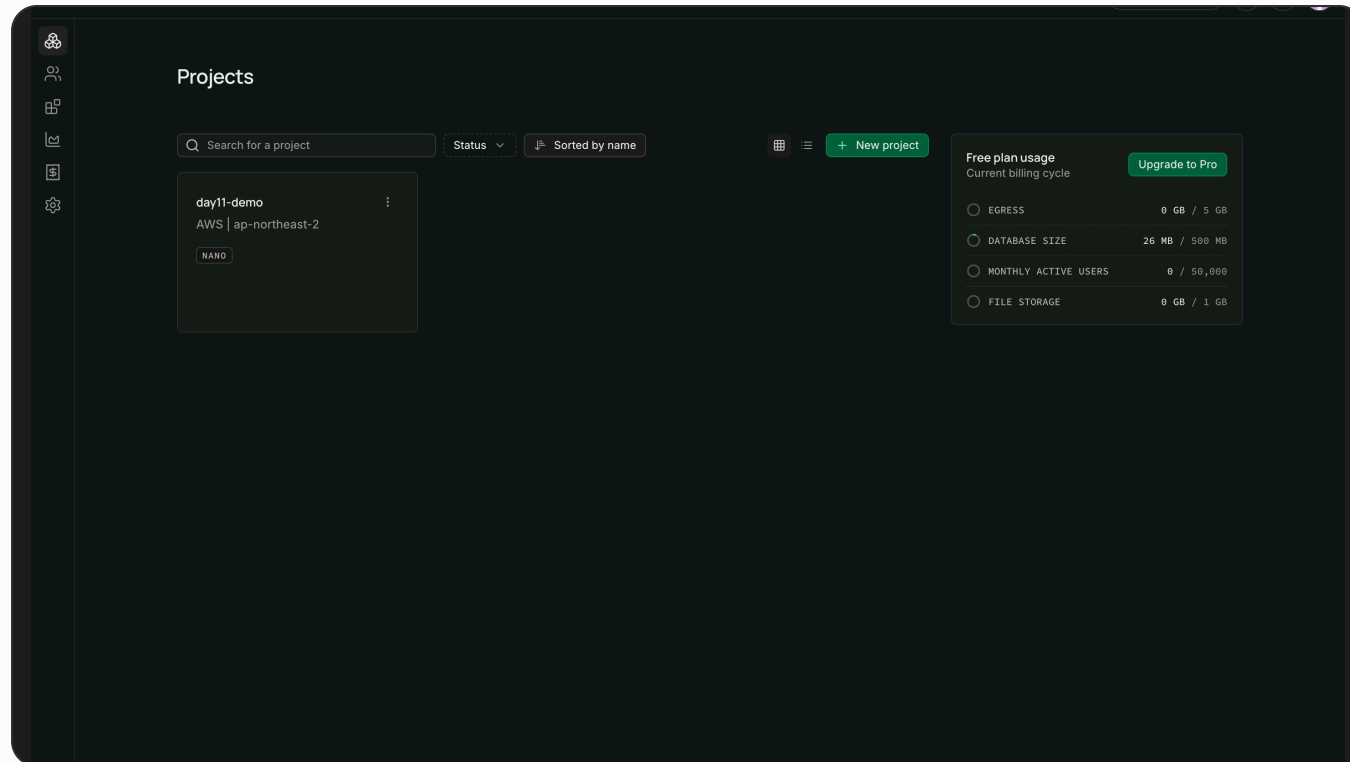
✓ 성공 표지: GitHub 인증 화면으로 넘어간다

왜 **GitHub 로그인** 이메일+비번 계정을 새로 안 만들어도 된다 - 이미 있는 GitHub 계정으로 로그인하면 계정 하나가 줄어든다.

막히면 GitHub 계정이 없으면: 이메일로 Sign up - 인증 메일 확인까지 시간이 더 걸린다

확인 GitHub 인증까지 마쳤으면 다음 -> **S1.2 · 장 8**

로그인이 끝나면 대시보드에서 "New project" 버튼을 찾는다



✓ 성공 표지: 오른쪽 위 초록색 "New project" 버튼이 보인다

왜 여기서 시작 Supabase는 프로젝트 단위로 데이터베이스를 하나씩 만든다
- 오늘 실습은 이 학생용 프로젝트 하나만 있으면 된다.

막히면 버튼이 안 보이면: 창을 조금 넓히거나 새로고침

확인 버튼을 봤으면 다음 -> **S2.1 · 장 9**

실캡처 2026-08-03 (Supabase 실제 프로젝트 목록, Aside 조회) - 이 계정은 이미 day11-demo 프로젝트가 있어 카드가 보인다. 처음 만드는 사람은 "아직 프로젝트가 없습니다" 빈 상태가 대신 보이며, New project 버튼 위치는 동일하다.

이름, 비밀번호, 리전을 채우고 "Create new project"를 클릭한다

Create a new project
Your project will have its own dedicated instance and full Postgres database.
An API will be set up so you can easily interact with your new database.

Organization: AIPM FREE

GitHub (optional): Connect GitHub
Ideal for agent-first workflows: update your schema in code, push it to GitHub, and Supabase deploys the changes automatically. [Learn more](#)

Project name: day11-demo

Database password: Copy
This password is strong. [Generate a password.](#)

Region: Asia-Pacific
Select the region closest to your users for the best performance.

Security:

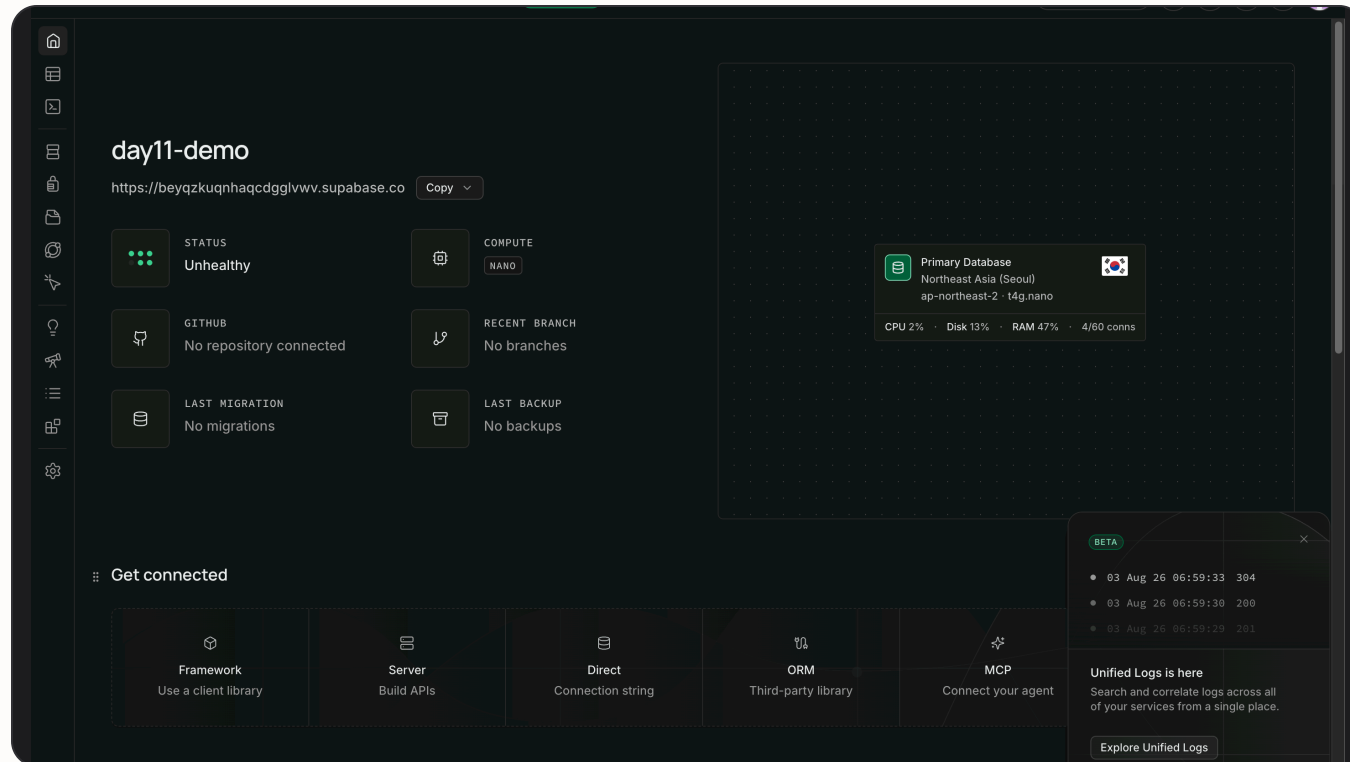
- ☒ **Enable Data API**
Autogenerate a RESTful API for your public schema.
Recommended if using a client library like [supabase-js](#).
- ☒ **Automatically expose new tables**
Grants privileges to Data API roles by default, exposing new tables.
We recommend disabling this to control access manually.
- ☐ **Enable automatic RLS**
Create an event trigger that automatically enables Row Level

이름은 뭐로 자유롭게 - 예시는 `teacher-apply` (선생님 신청). 비밀번호는 이 프로젝트의 DB 관리자 비번 - 비밀번호 관리자에 저장해 둔다. 리전은 한국에서 가까운 Northeast Asia(Seoul) 권장 - 응답이 더 빠르다.

막히면 비밀번호 규칙에 걸리면: 화면에 뜨는 조건(길이·특수문자)을 그대로 따른다

확인 Create를 눌렀으면 다음 -> S2.2 · 장 10

"Setting up project..." 화면에서 기다린다



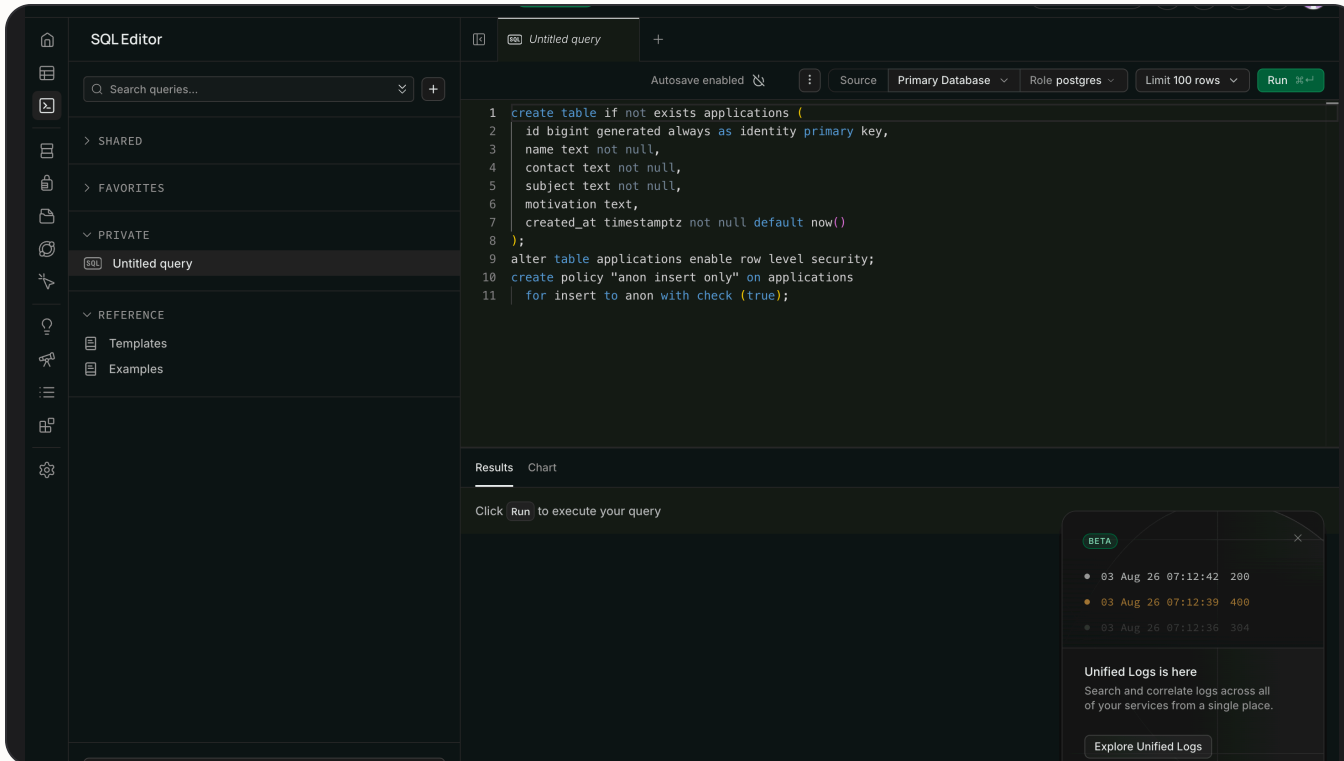
왜 기다리나 Supabase가 이 프로젝트 전용 데이터베이스 서버를 새로 만드는 중이다. 약 2분 걸리는 것이 정상 - 다른 학생이 먼저 끝나도 조금해하지 않는다.

180초 규칙 3분을 넘게 이 화면이면: 캡처 + 채팅 "2번 막힘"

확인 프로젝트 대시보드로 넘어갔으면 다음 -> S3.1 · 장 11

실캡처 2026-08-03 (Supabase 실제 초기화 중 화면, Aside 조회) - 이 계정 화면은 문구 대신 STATUS "Unhealthy"·Success Rate 0.0%로 초기화 중임을 표시한다(버전에 따라 "Setting up project..." 문구 화면이 나올 수도 있음) - 어느 쪽이든 잠시 기다리면 정상으로 바뀐다.

왼쪽 메뉴 SQL Editor를 열고 setup.sql 전체를 붙여넣는다

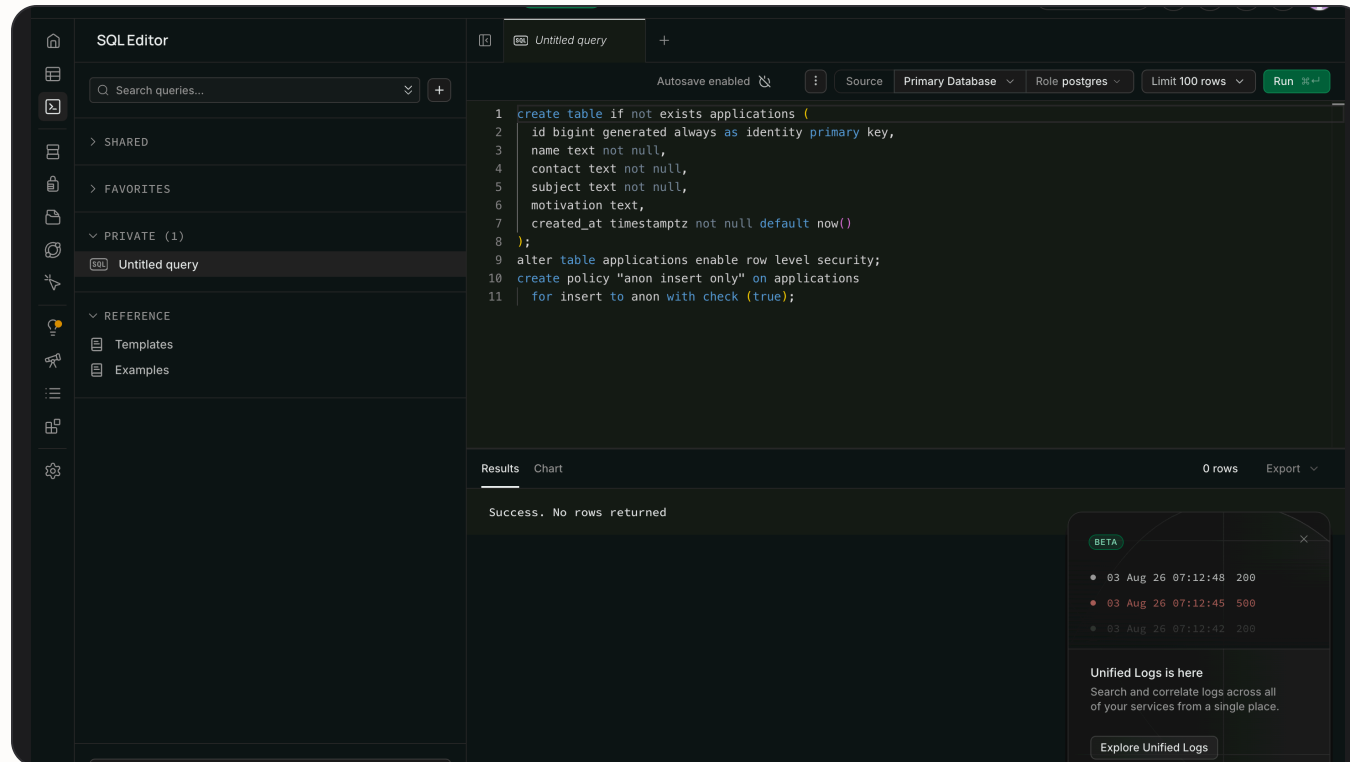


왜 통째로 이 파일 하나가 오늘 쓸 표(applications)의 설계도다. 표 구조(컬럼)와 "익명은 넣기만 가능"이라는 보안 규칙(RLS)이 한 번에 실행된다.
motivation 만 not null 이 없다 - 선택 문항이라 비워도 되기 때문이다.

막히면 setup.sql 파일을 못 찾으면: Slack #자료공유의 파일 또는 메시지 텍스트 본 사용

확인 붙여넣었으면 다음 -> **S3.2 · 장 12**

오른쪽 위 "Run" 버튼을 클릭하고 "Success"를 확인한다



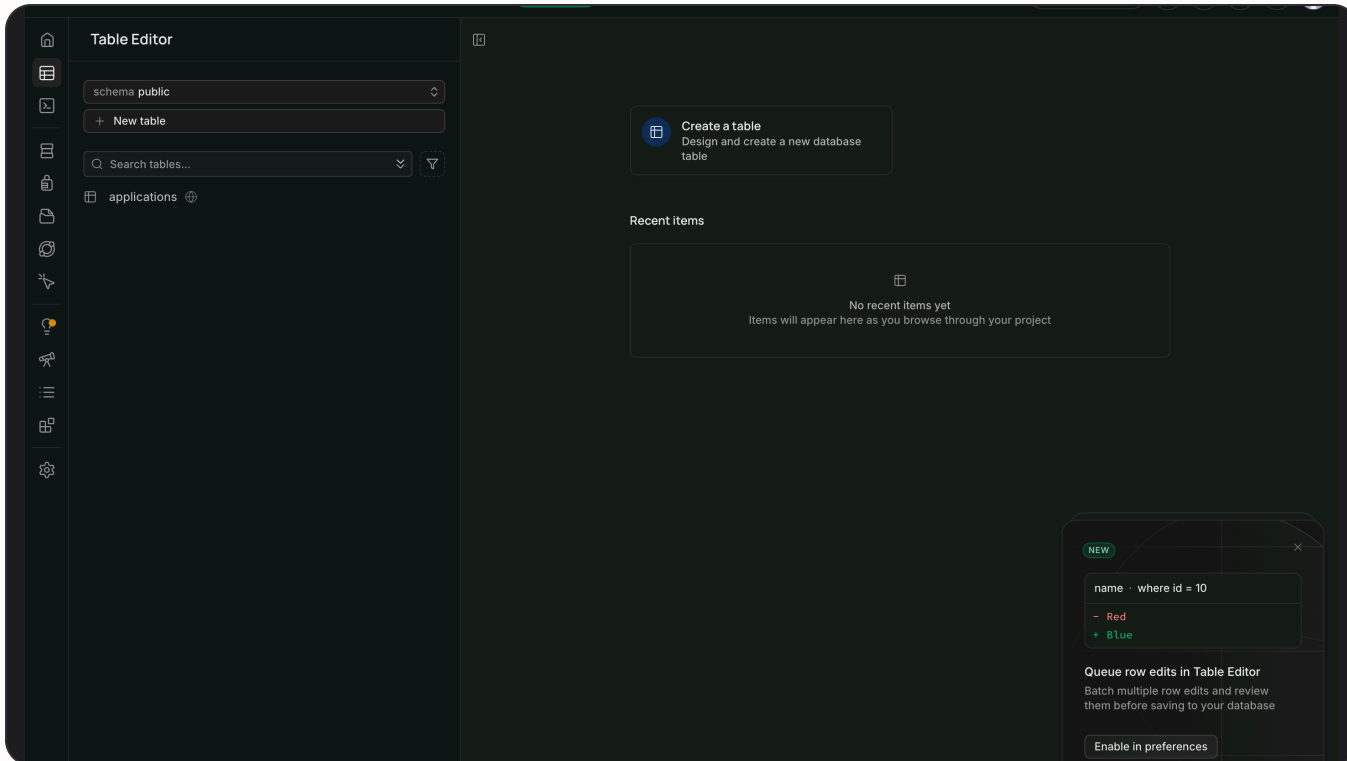
✓ 성공 표지: 초록색 "Success" 배너

왜 이 확인이 중요한가 여기서 실패를 놓치고 넘어가면, 나중에 폼을 제출했을 때 `relation applications does not exist` 에러로 되돌아온다. 지금 확인하는 것이 가장 싸다.

막히면 빨간 에러가 뜨면: 에러 문구 전체를 캡처해 채팅 "3번 막힘" - setup.sql 을 다시 전체 복사해 재시도

확인 Success를 봤으면 다음 -> S4.1 · 장 13

왼쪽 메뉴 Table Editor를 열어 applications가 생겼는지 본다



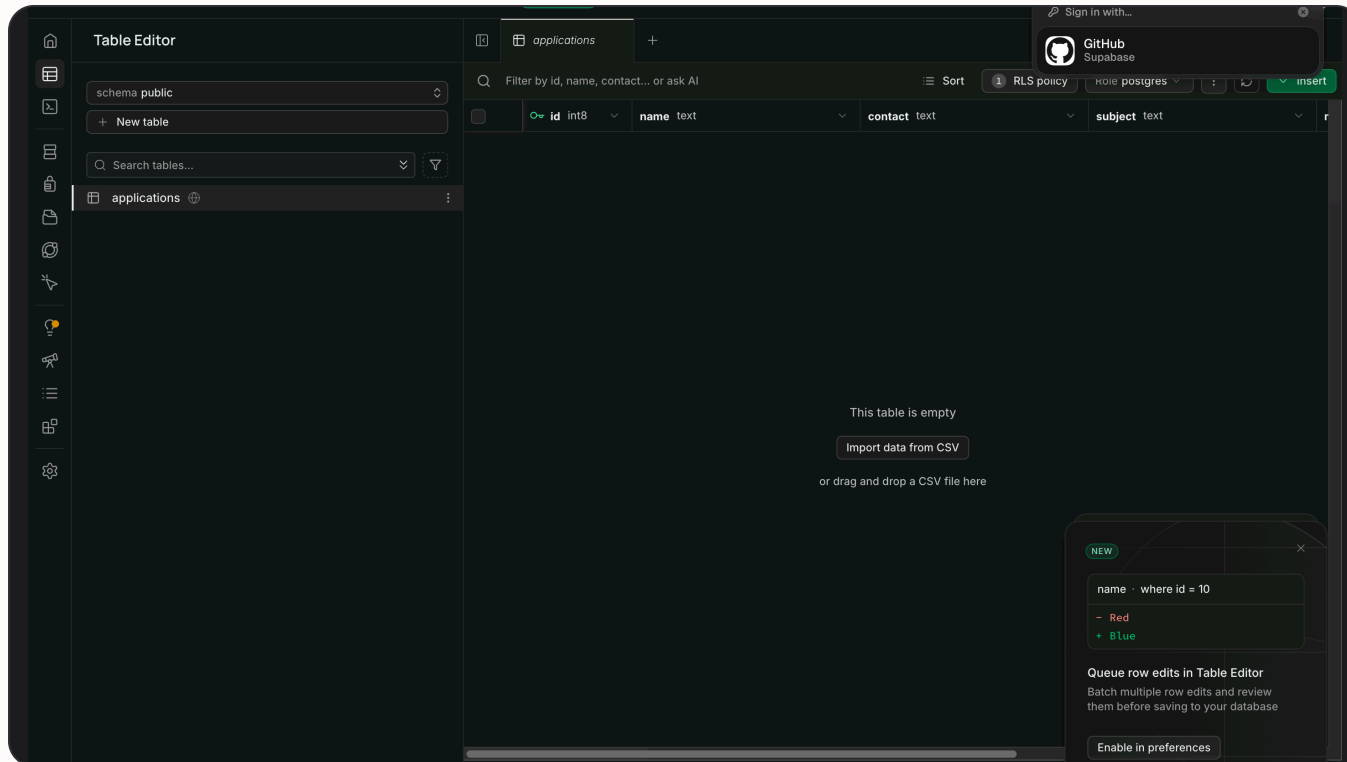
✓ 성공 표지: 표 목록에 **applications** 이름이 보인다

왜 눈으로 다시 확인 S3의 "Success"는 실행 성공이고, 여기서 보는 것은 **결과 물이 실제로 생겼는가**다 - 확인 지점을 하나 더 두면 실수를 더 빨리 잡는다.

막히면 목록이 비어 있으면: S3을 다시 실행했는지 확인 - 다른 프로젝트를 보고 있을 수도 있다(좌상단 프로젝트 이름 확인)

확인 applications를 봤으면 다음 -> **S4.2 · 장 14**

applications를 클릭해 열고 컬럼을 확인한다



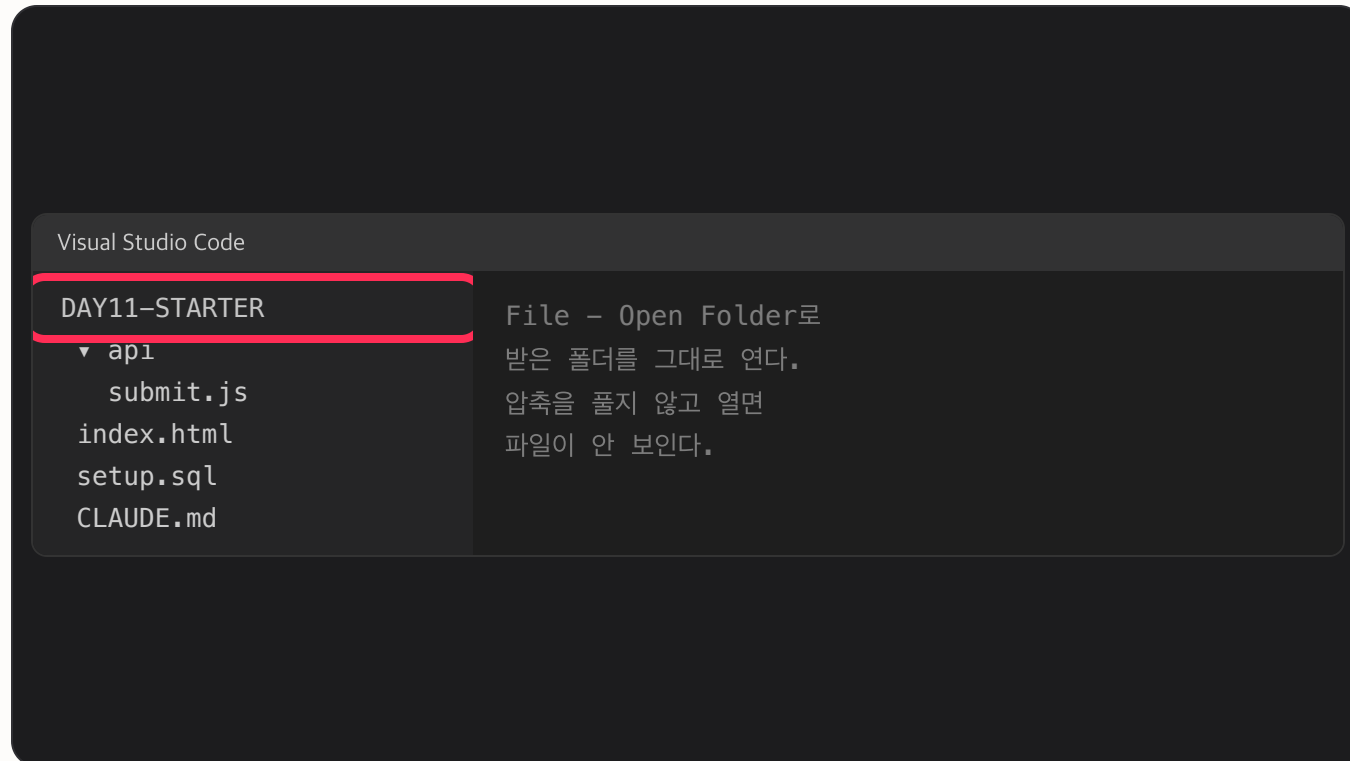
왜 지금 컬럼만 확인 행이 0개인 것은 문제가 아니다 - 지금 볼 것은 딱 하나, setup.sql이 만든 **칸의 이름**(name/contact/subject/motivation)이 폼 필드와 1:1로 맞는가다.

막히면 컬럼 이름이 다르면: S3을 다시 확인 - setup.sql 원문과 다른 것을 붙여 넣었을 수 있다

확인 컬럼을 봤으면 다음 -> **S5.1 · 장 15**

실캡처 2026-08-03 (Supabase 실제 Table Editor, applications 테이블을 연 화면) - id/name/contact/subject 컬럼이 화면에 보이고, motivation/created_at은 오른쪽으로 스크롤하면 이어진다. "This table is empty · 0 records"로 0행 상태가 실물로 확인된다.

Slack에서 받은 starter 폴더를 VS Code로 연다



✓ 성공 표지: 왼쪽 목록에 index.html / api / setup.sql이 보인다

왜 이 파일들인가 index.html(신청 폼 화면), api/submit.js(저장 서버 코드), setup.sql(표 설계도) 세 파일이 오늘 다룰 전부다.

막히면 파일이 하나도 안 보이면: zip이 안 풀렸을 수 있다 - 압축 해제 후 그 폴더를 다시 연다

확인 목록을 봤으면 다음 -> **S5.2 · 장 16**

터미널을 열고 npm i를 실행한다

```
teacher-apply - -zsh - 80x24

user@dudu teacher-apply % npm i
(패키지 목록을 읽고 설치하는 줄이 몇 줄 지나가고)
added N packages, and audited N+1 packages in Ns

found 0 vulnerabilities
user@dudu teacher-apply %
```

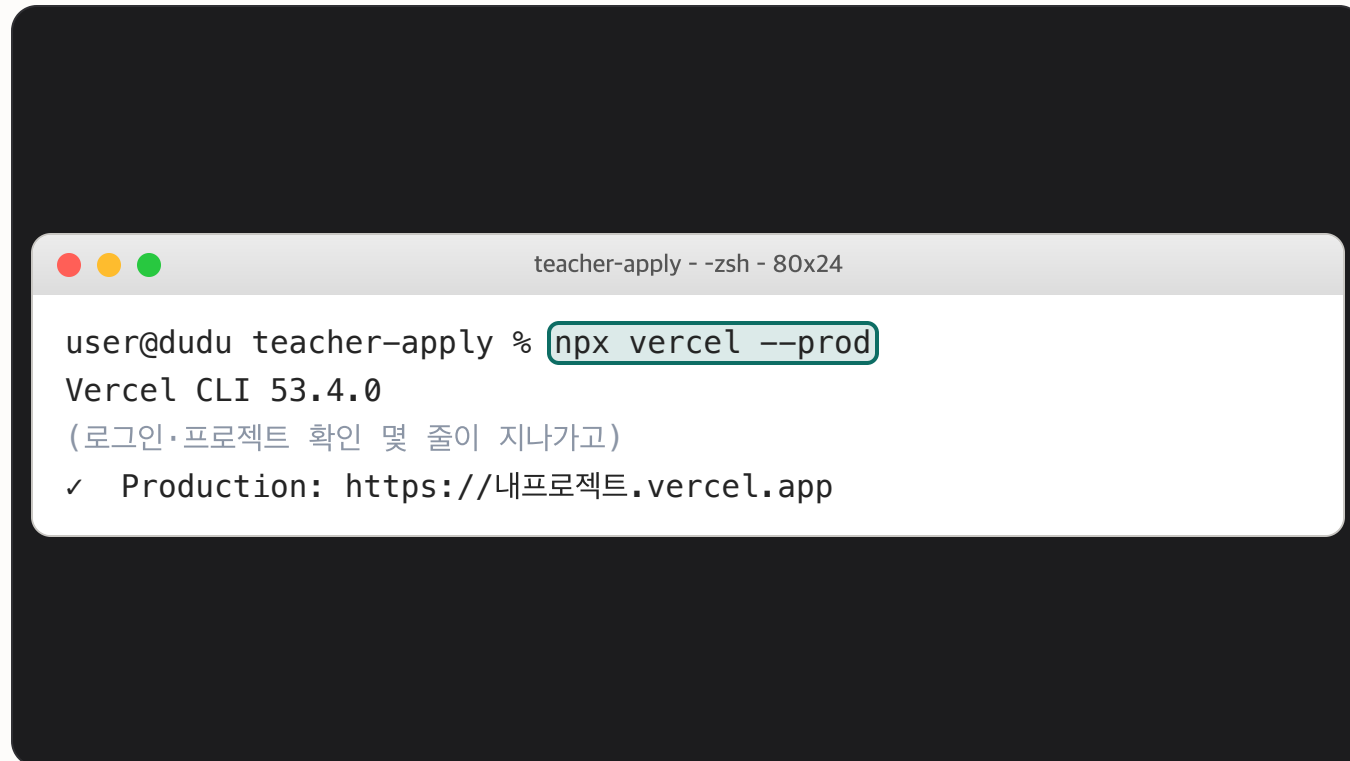
왜 이번엔 패키지 이름이 없나 Day9는 `npm i @vercel/analytics` 처럼 이름을 직접 적었다. 오늘은 package.json에 `@supabase/supabase-js` 가 이미 적혀 있어서, 이름 없이 `npm i` 만 실행하면 그 목록을 읽어 전부 설치한다.

✓ 성공 표지: added N packages 줄 (정확한 숫자·초 수는 컴퓨터마다 달라도 된다)

막히면 에러가 나면: 마지막 3줄을 검색 - Day9 에러 전화번호부의 E-NPM/E-POLICY 계열과 같은 원인이다

확인 added 줄을 봤으면 다음 -> **S6.1 · 장 17**

터미널에 `npx vercel --prod`를 입력하고 Enter



```
teacher-apply - zsh - 80x24

user@dudu teacher-apply % npx vercel --prod
Vercel CLI 53.4.0
(로그인·프로젝트 확인 몇 줄이 지나가고)
✓ Production: https://내프로젝트.vercel.app
```

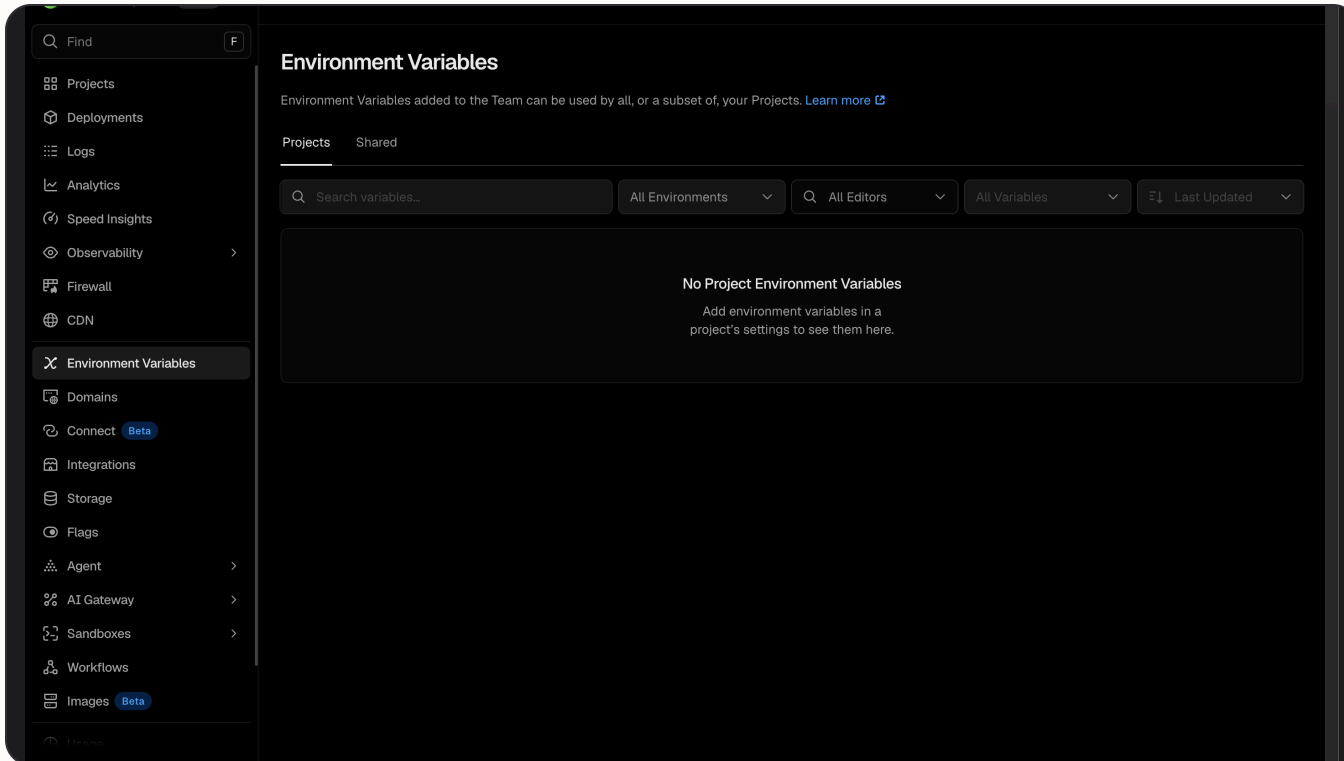
같은 명령, 같은 도구 Day9에서 쓴 배포 명령과 완전히 같다 - Vercel은 이 학생 프로젝트가 처음이라 몇 가지를 더 묻는다(프로젝트 연결 여부 등). 화면 안내를 그대로 따른다.

✓ **성공 표지: 마지막에 Production: <https://...vercel.app> 주소**

막히면 로그인 화면(코드 입력)이 뜨면 에러 아님 - Day9 E-AUTH-001과 동일. 그 외 에러는 마지막 3줄 캡처 + "6번 막힘"

확인 주소를 봤으면 다음 -> **S7.1 · 장 18**

Vercel 프로젝트 Settings -> Environment Variables에서 SUPABASE_URL을 넣는다

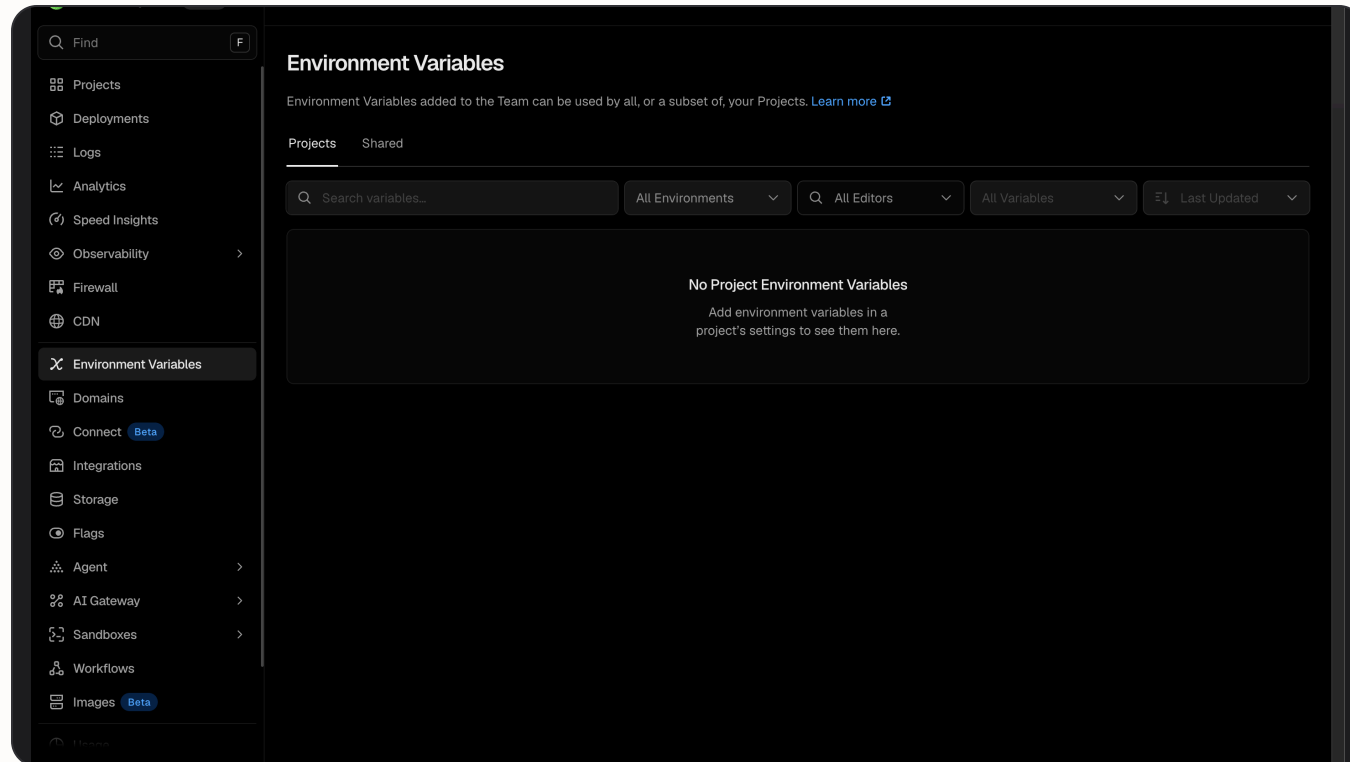


어디서 복사하나 자기 Supabase 프로젝트 Settings -> API 화면의 Project URL 값을 그대로 복사해 온다. 값은 매번 다르다 - 위 xxxx 는 예시일 뿐이다.

막히면 Project URL을 못 찾으면: Supabase 왼쪽 메뉴 맨 아래 Settings -> API

확인 입력했으면 다음 -> **S7.2 · 장 19**

같은 화면에 SUPABASE_ANON_KEY를 추가한다 - anon public 값만



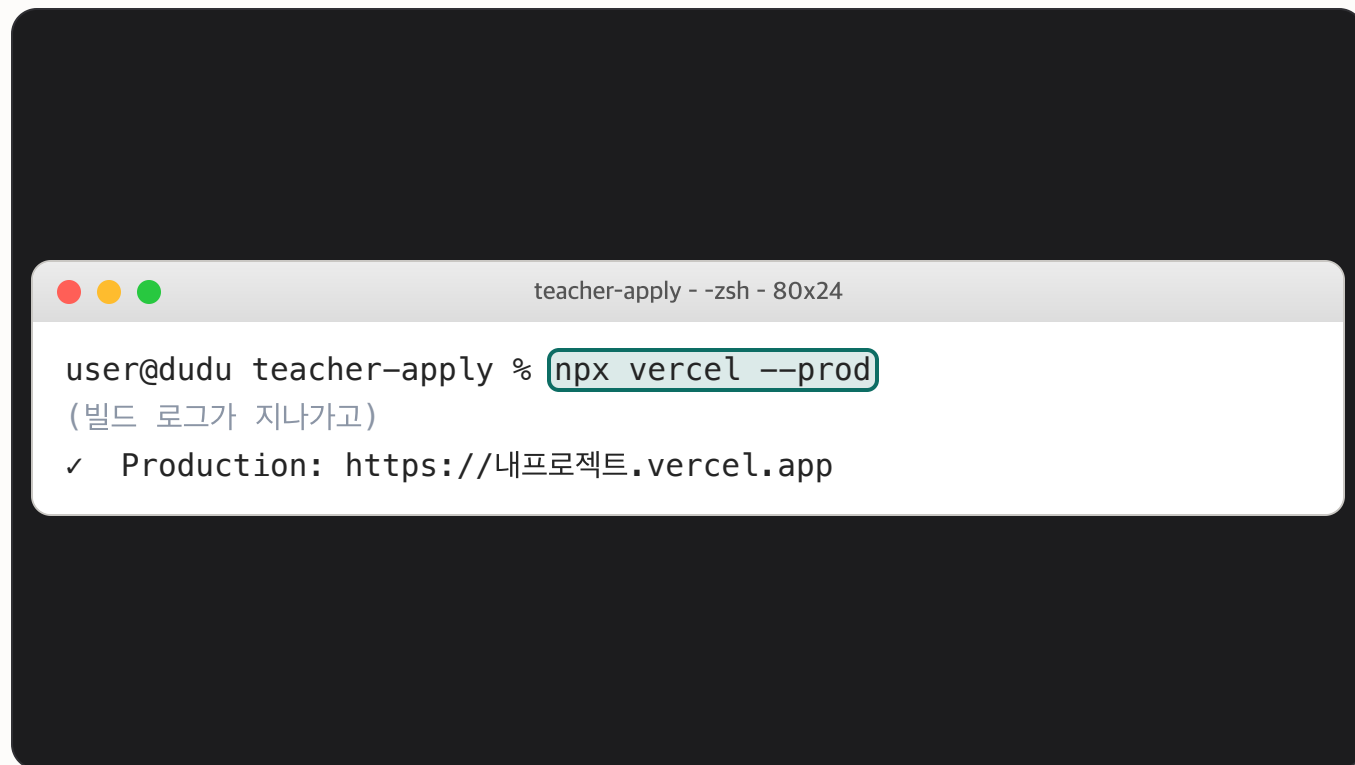
anon public만 Supabase Settings -> API 화면엔 값이 두 개 보인다 - **anon public만** 쓴다. `service_role` 은 관리자 전용 키라 이 실습에서 쓰면 보안 사고다.

절대 규칙 이 두 값을 채팅·Slack·화면공유에 **붙여넣지 않는다**. 각자 자기 화면에서 자기 값만 입력한다.

확인 Save까지 눌렀으면 다음 -> **S8.1 · 장 20**

실캡처 2026-08-03 (Vercel 팀 레벨 Environment Variables 화면, Aside 조회) - 같은 화면 재사용. 실제 키 값(eyJ...)은 화면에 없으며, 화면 공유 금지 규칙은 왼쪽 안내 그대로 유효하다.

터미널로 돌아와 npx vercel --prod를 한 번 더 실행한다



```
teacher-apply - -zsh - 80x24
user@dudu teacher-apply % npx vercel --prod
(빌드 로그가 지나가고)
✓ Production: https://내프로젝트.vercel.app
```

왜 또 배포하나 방금 넣은 환경변수는 **다음 배포부터** 적용된다. S6.1에서 이미 배포했어도, 환경변수는 그 이후에 넣었으니 한 번 더 배포해야 서버가 그 값을 읽는다.

막히면 S6.1과 같은 명령이다 - 에러가 나면 그 장의 에러코드를 참고

확인 Production 주소가 다시 찍혔으면 다음 -> **S9.1 · 장 21**

새 탭에서 내 배포 주소를 열고, 내가 직접 신청서 1건을 제출한다

← → ↻ 내프로젝트.vercel.app

선생님 신청하기

지도 가능한 과목과 연락처를 남겨주시면...

김민수

minsu@example.com

코딩 실습

신청서 제출하기

왜 내가 직접 Day9에서 확인한 것과 같은 이유 - 남이 넣어주길 기다리면 "데이터가 안 오는 것"과 "아무도 안 온 것"을 구분할 수 없다.

막히면 배포 주소가 안 열리면: S8.1의 Production 주소를 다시 확인

확인 "신청서가 접수되었습니다"를 봤으면 다음 -> **S9.2 · 장 22**

Supabase Table Editor로 돌아와 applications를 새로고침한다

teacher-apply

id	name	contact	subject	motivation
1	김민수	minsu@example.com	coding	코딩 교육에 관심이 많습니다

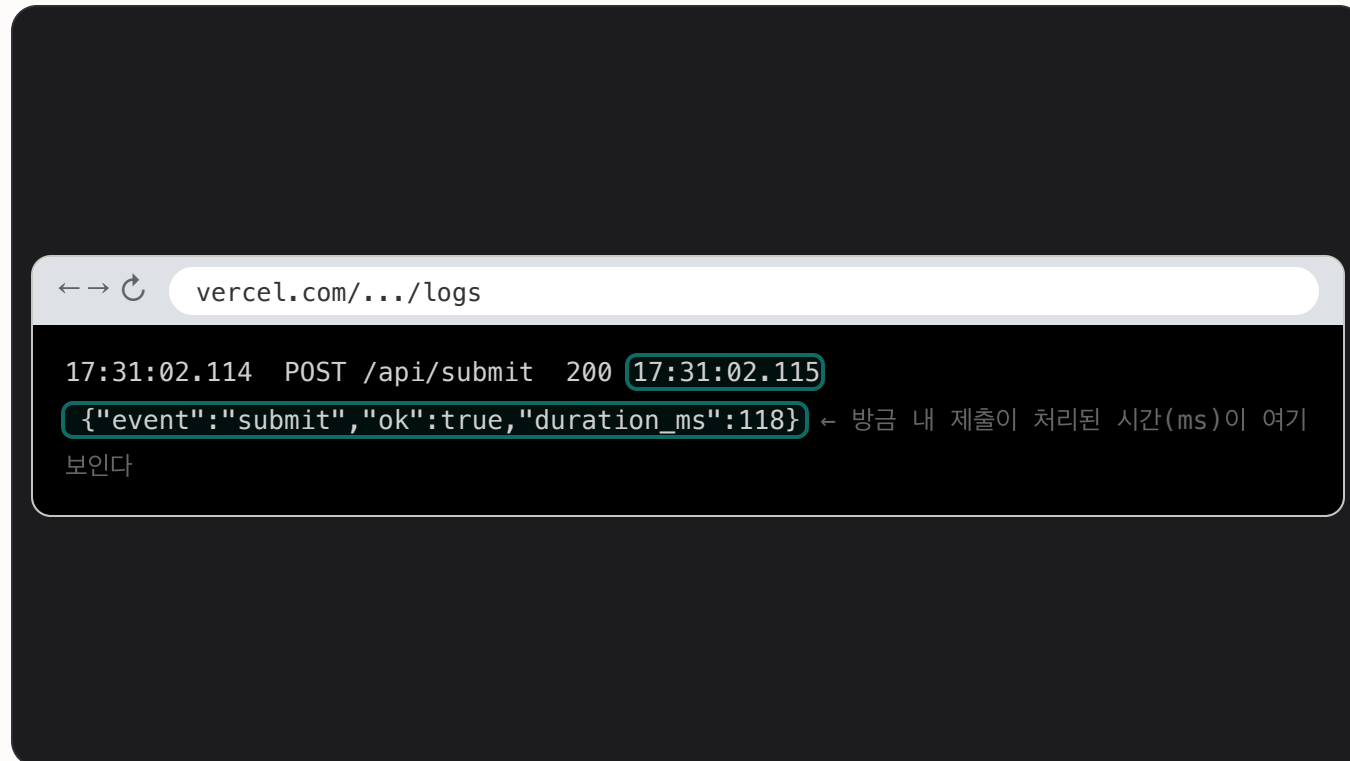
✓ 성공 표지: 방금 넣은 값 그대로 행 1개. 터미널의 "완료"가 아니라 이 행이 오늘의 성공이다

안 보이면 Day9의 그 문장이 오늘도 적용된다 - "완료 메시지"는 발송 확인이지 도착 확인이 아니다. 도착은 내 눈으로 확인한다. 순서대로 쭈뼛다: (1)S3 setup.sql 실행 여부 (2)S7 환경변수 오타 (3)S8 재배포 여부.

막히면 행이 안 보이면: 위 3가지를 순서대로 확인 + 캡처 + 채팅 "9번 막힘"

확인 내 행을 봤으면 다음 -> **S9.3 · 장 23**

Vercel Logs 탭에서 방금 요청의 응답 시간(duration_ms)까지 읽는다



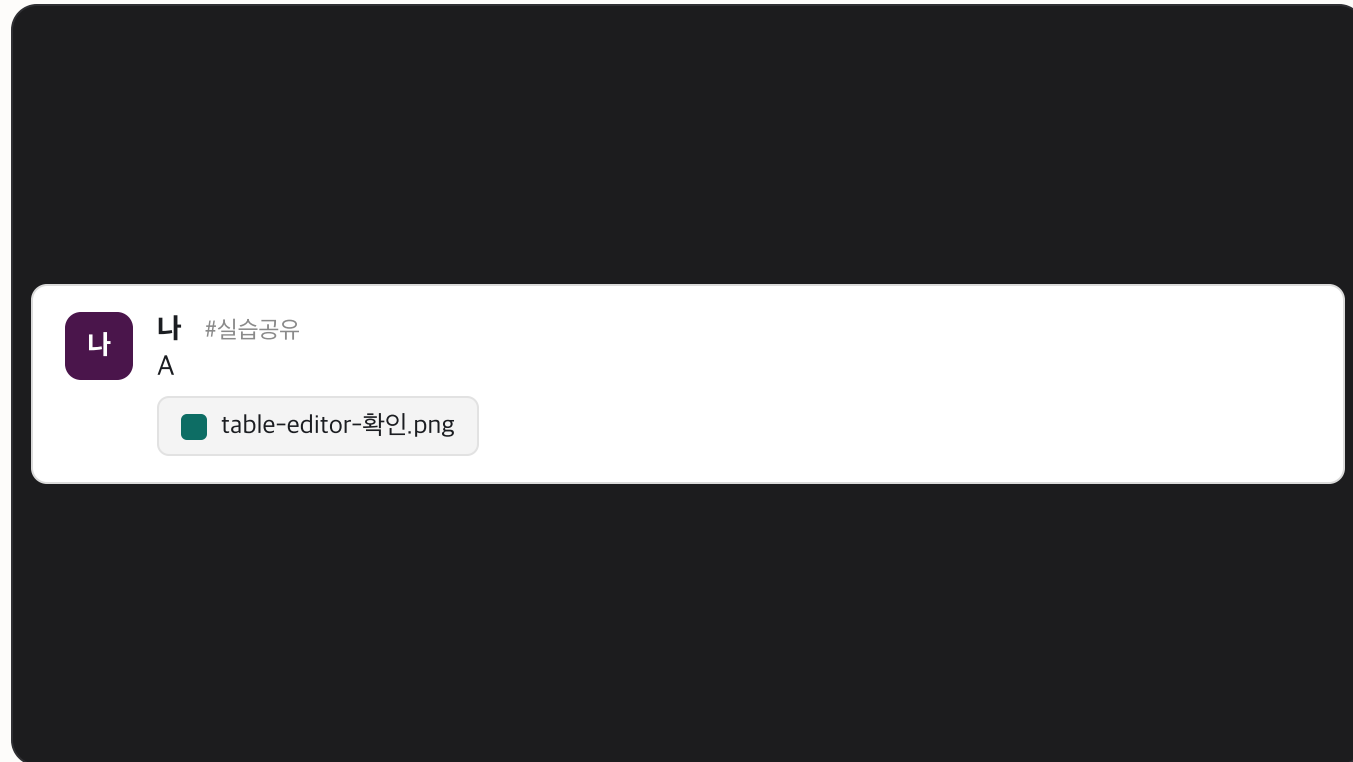
왜 시간까지 보나 S9.2에서 확인한 것은 "요청이 도착했는가"였다. 오늘은 한 걸음 더 - "그 요청이 얼마나 걸렸는가"까지 같은 로그에서 읽는다.
api/submit.js가 처리 시작 시각(`t0`)과 끝난 시각의 차이를 `duration_ms` 라는 이름으로 로그 한 줄에 직접 남기게 만들었기 때문이다.

✓ **성공 표지: POST /api/submit 200 줄 바로 아래**
`{"event": "submit", "ok": true, "duration_ms": ...}` 줄

막히면 `duration_ms` 줄이 안 보이면: api/submit.js에 `console.log(JSON.stringify(...))` 줄이 있는지, S8 재배포까지 했는지 확인

확인 `duration_ms` 줄을 봤으면 다음 -> **S9.4 · 장 24**

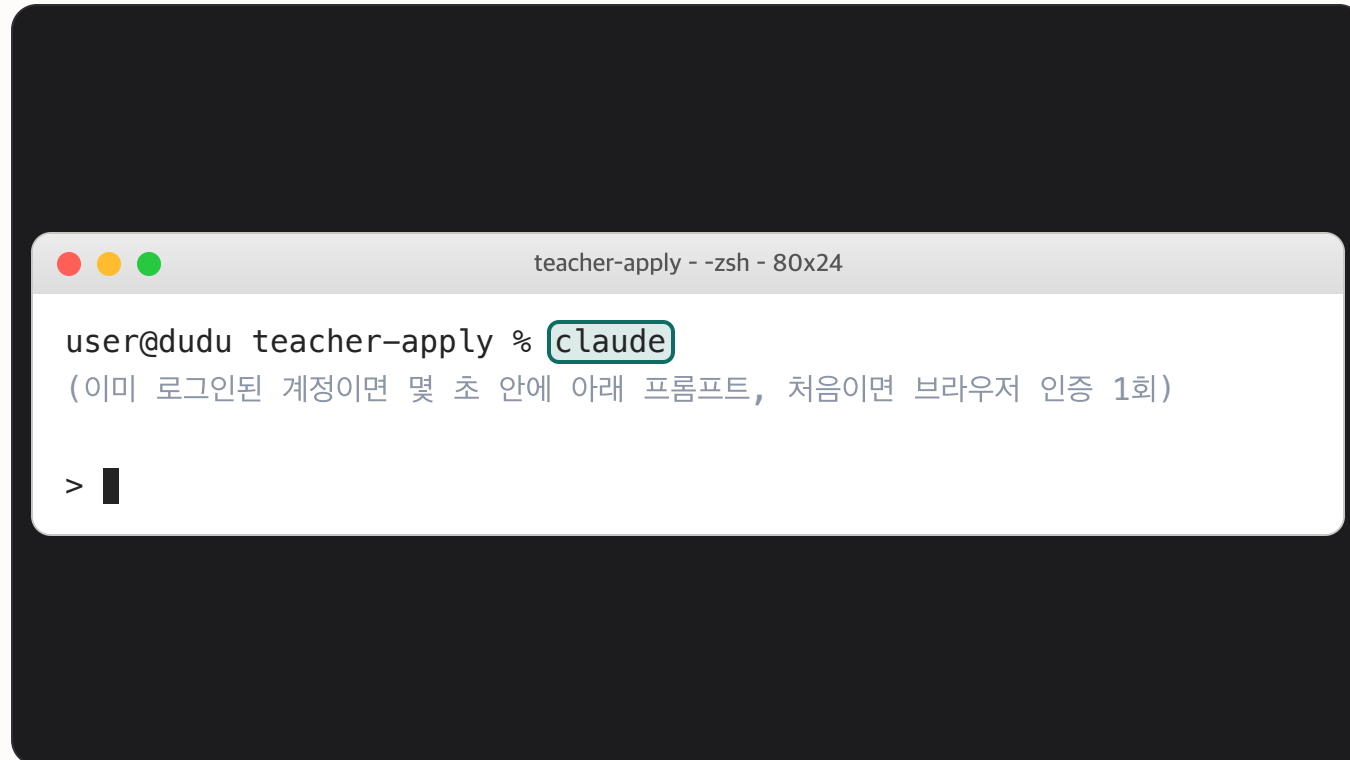
#실습공유에 "A" + Table Editor 캡처를 올린다



왜 캡처까지 "됐어요"라는 말보다 화면 캡처 한 장이 더 빠르고 정확한 증거다
- 강사도 이 캡처로 전체 진행 상황을 한눈에 본다.

여기까지가 오늘의 절반 - 지금부터는 Claude Code로 신청 폼에 질문 하나를 더 추가해 본다. 다음 -> **S10.1 · 장 25**

같은 프로젝트 폴더 터미널에서 claude를 실행하고, 로그인 상태를 확인한다



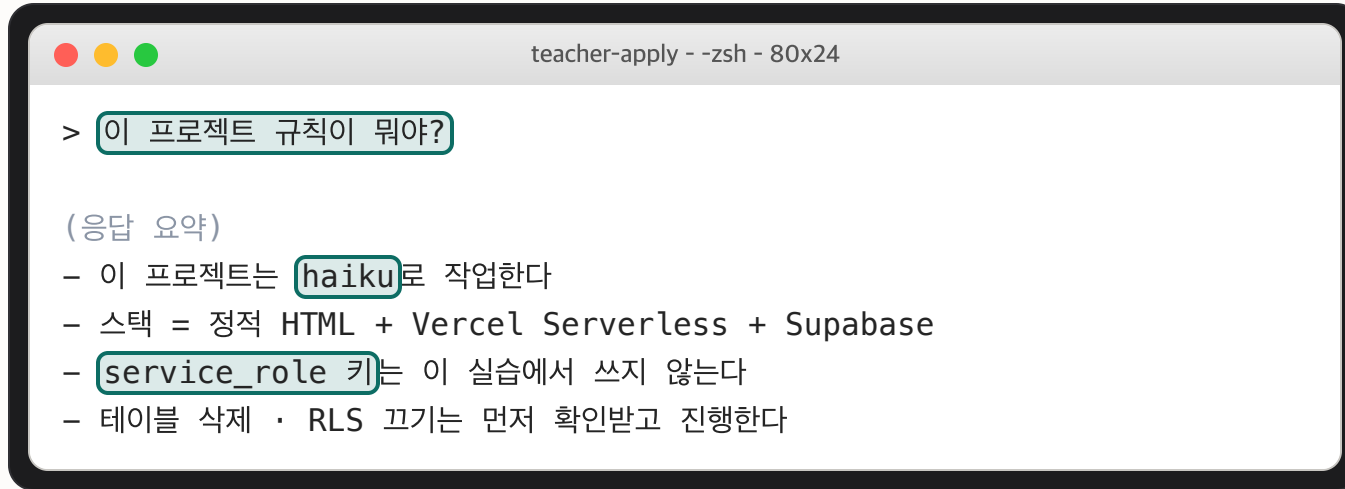
왜 모델을 안 고르나 `.claude/settings.json` 에 `model: haiku` 가 이미 고정돼 있다 - 이 세션은 자동으로 haiku로 시작한다. Day9의 GitHub 로그인과 같은 원리 - 계정 인증은 한 번만 하면 된다.

✓ **성공 표지:** > 프롬프트가 뜬다 (대화 입력 대기 상태)

막히면 브라우저 인증 창이 안 뜨면: 터미널을 껐다 다시 열어 같은 폴더에서 재 시도

확인 프롬프트를 봤으면 다음 -> **S10.2 · 장 26**

"이 프로젝트 규칙이 뭐야?"라고 물어, CLAUDE.md가 읽혔는지 확인한다



```
teacher-apply - zsh - 80x24

> 이 프로젝트 규칙이 뭐야?

(응답 요약)
- 이 프로젝트는 haiku로 작업한다
- 스택 = 정적 HTML + Vercel Serverless + Supabase
- service_role 키는 이 실습에서 쓰지 않는다
- 테이블 삭제 · RLS 끄기는 먼저 확인받고 진행한다
```

왜 이 확인이 중요한가 CLAUDE.md는 폴더 안의 규칙 파일이다 - Claude Code는 대화를 시작할 때 이 파일을 먼저 읽는다. 그 증거가 방금 응답 안의 `haiku` · `service_role` 같은 이 프로젝트만의 단어들이다. 이 확인을 건너뛰면, 나중에 규칙이 안 지켜져도 원인을 못 잡는다.

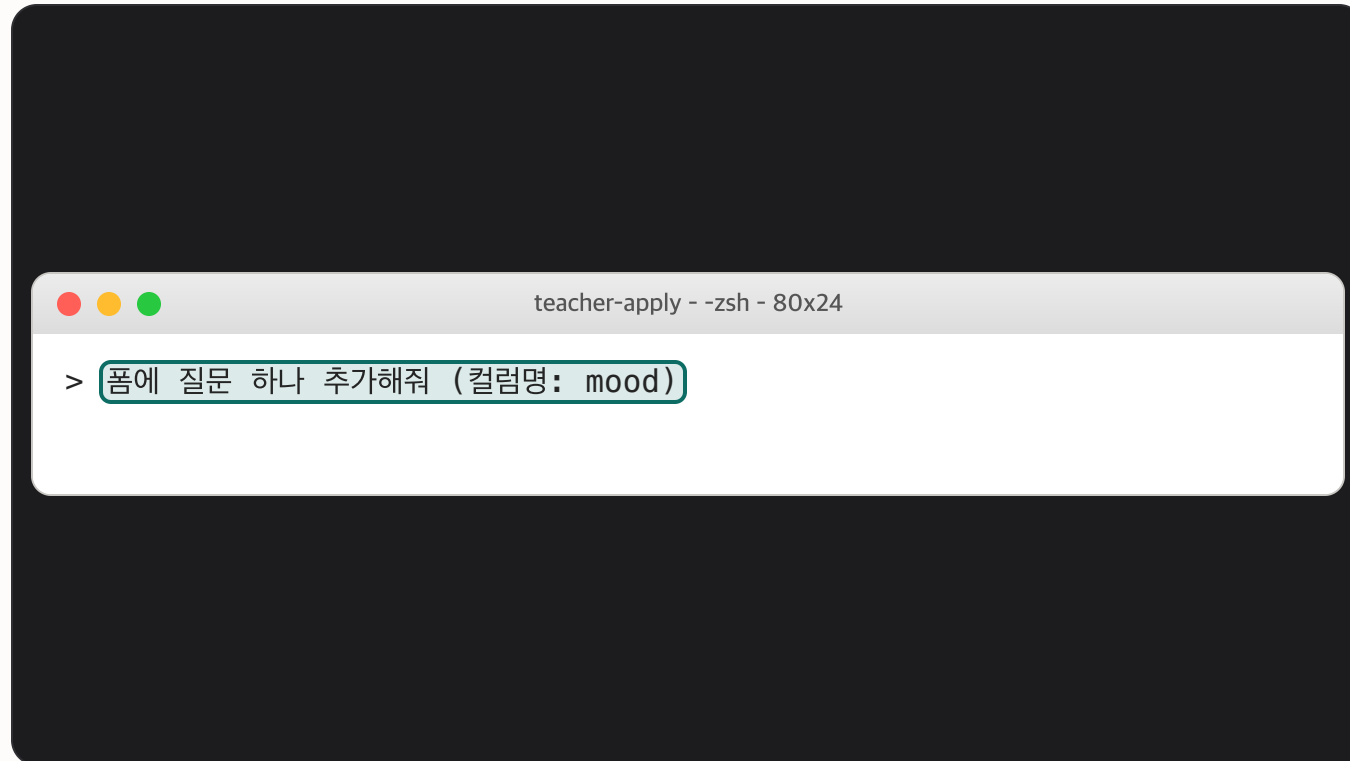
✓ **성공 표지: 응답에 이 프로젝트만의 단어(haiku, service_role, RLS 등)가 등장한다**

막히면 일반적인 답만 나오면: CLAUDE.md 파일이 프로젝트 폴더 맨 바깥에 있는지 확인

확인 규칙 언급을 봤으면 다음 -> **S10.3 · 장 27**

Claude Code 실제 화면 구성은 다를 수 있다 - 여기서는 대화 흐름 개념만 표현했다. 실제 응답 문구는 매번 다를 수 있으나, CLAUDE.md에 적힌 규칙이 근거로 등장한다는 개념은 동일하다.

이제 원하는 것을 문장으로 말한다 - "폼에 질문 하나 추가해줘 (컬럼명: mood)"



왜 하필 mood인가 오늘 실습의 목적은 "실제로 쓸 문항 설계"가 아니라 "요청 한 문장이 파일 세 곳을 함께 바꾼다"는 구조를 보는 것이다. mood는 지원서 설계와 무관한 연습용 예시 컬럼이다. **실제 이름**(컬럼명)을 문장에 함께 주면, 화면 문구와 저장되는 이름이 같리지 않는다 - 선택 박스의 화면 글자와 token이 다르다고 배운 것과 같은 이치다.

✓ 성공 표지: Claude가 "세 파일을 함께 고치겠다"고 안내한다

막히면 claude 명령이 안 먹으면: VS Code 터미널이 이 프로젝트 폴더에서 열려 있는지 확인(S10.1과 같은 확인)

확인 안내를 봤으면 다음 -> **S10.4 · 장 28**

질문 하나가 파일 세 곳을 함께 바꾼다 - 폼, 저장 코드, 표 설계도

```
<label for="subject">지도 가능 과목</label>
... (기존 선택트 박스) ...
<label for="mood">오늘 기분</label>
<input id="mood" name="mood">
```

```
const { name, contact, subject, mood } = req.body || {};
...
.insert({ name, contact, subject, mood });
```

```
alter table applications add column mood text;
```

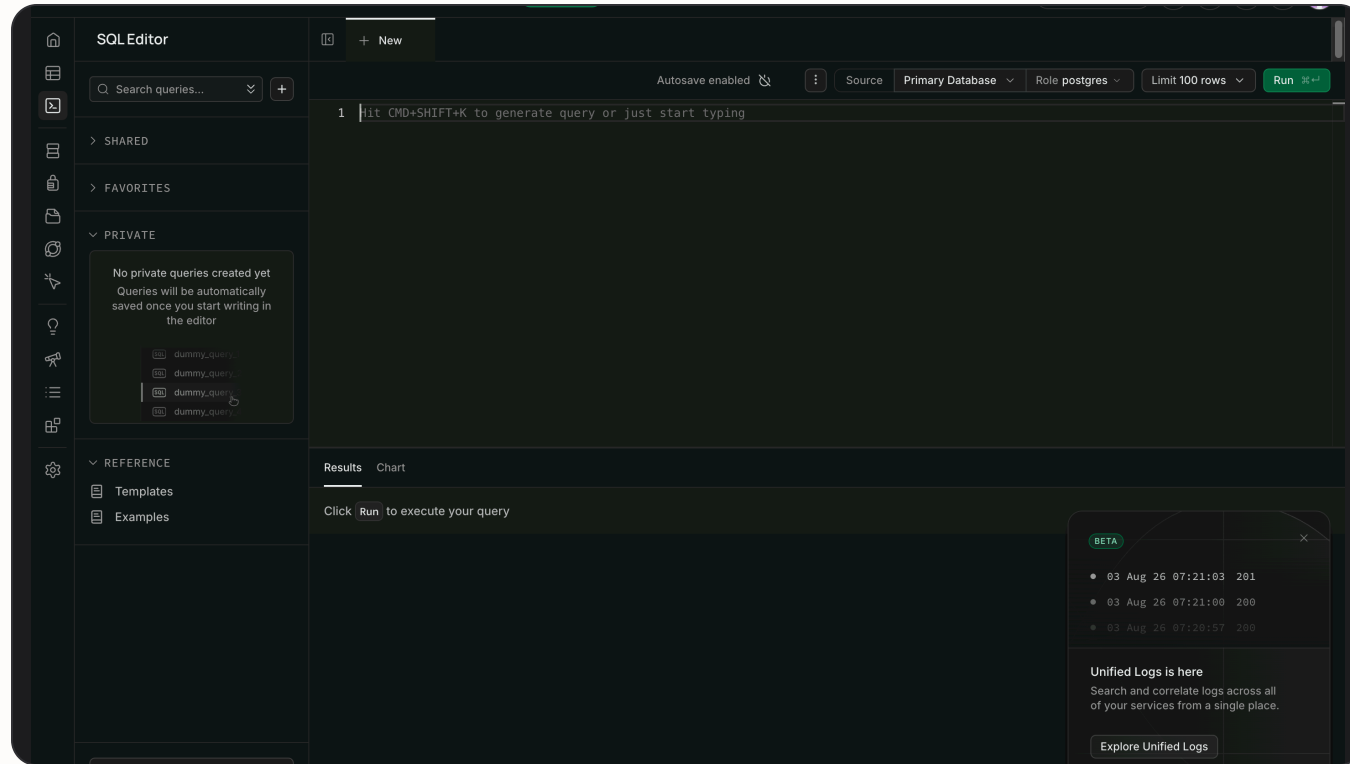
왜 세 곳 다 바뀌나 화면(index.html)만 바뀌면 값을 보내도 서버가 안 받는다. 서버(submit.js)만 바뀌면 화면에 입력칸이 없다. 표(setup.sql)까지 바뀌어야 저장할 칸이 생긴다 - 셋은 항상 **한 세트**다.

✓ 성공 표지: 세 파일에서 mood라는 같은 이름이 동시에 보인다

막히면 파일 하나만 바뀌었으면: Claude에게 "세 파일 다 바뀌었는지 확인해 줘"라고 다시 요청

확인 세 파일을 봤으면 다음 -> **S10.5 · 장 29**

표 구조 변경은 Supabase에서 직접, 나머지는 /deploy 한 마디로



```
alter table applications add column mood text;
```

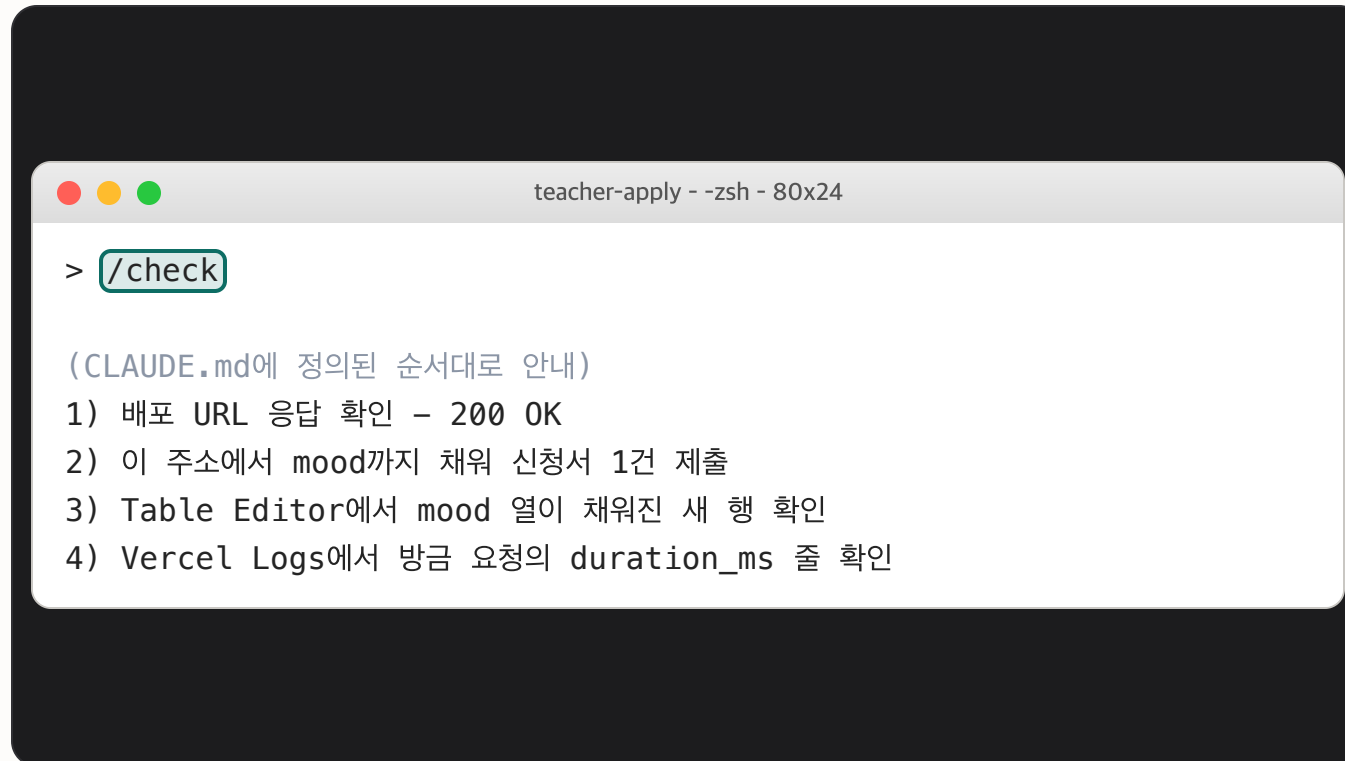
왜 Claude가 표까지 못 바꾸나 Claude Code는 내 컴퓨터의 파일 (index.html, submit.js, setup.sql)만 고친다. 실제 표 구조를 바꾸는 것은 Supabase 서버 안의 일이라, setup.sql에 적힌 alter문을 **SQL Editor에서 내가 직접 Run**해야 한다. 이 지점이 오늘 에이전트가 못 건드리는 유일한 곳이다.

이어서 할 일 Run 확인 -> 터미널로 돌아와 Claude에게 `/deploy` 요청 -> CLAUDE.md에 정의된 순서(비밀값 검사 -> `git add·commit` -> `npx vercel --prod`)가 그대로 실행된다

확인 Production 주소가 다시 찍혔으면 다음 -> **S10.6 · 장 30**

실캡처 2026-08-03 (Supabase 실제 SQL Editor, day11-demo 프로젝트의 새 쿼리 빈 화면) - 위 화면은 실제 빈 SQL Editor이고, 아래 코드 상자는 여기에 직접 입력할 alter문이다(자동 입력이 아니라 학생이 직접 타이핑). /deploy 실행 장면은 다음 장에서 개념적으로 이어진다.

/check 한 마디로 Day9부터 이어진 검증 순서를 다시 밟는다



터미널 완료 != 성공, 오늘도 동일 /check 는 확인할 순서를 대신 불러줄 뿐이다. 실제로 화면을 열어 눈으로 보는 것은 오늘도 사람이다 - Day9부터 이어지는 원칙이 명령 하나로 자동 상기된다.

✓ 성공 표지: mood 값까지 채워진 새 행을 Table Editor에서 내 눈으로 본다

막히면 안내된 4단계 중 하나라도 안 되면: 그 번호를 짚어 "3번이 안 돼"처럼 구체적으로 다시 요청

확인 mood 값까지 확인했으면 오늘 실습 종료 -> [집계 · 장 31](#)

#실습공유에 A/B/C 중 하나로 전원 채팅

표기	뜻	같이 올릴 것
A	applications에서 내 행을 확인했다	Table Editor 캡처(또는 Logs duration_ms 캡처)
B	배포까지는 했지만 행 확인 전	Production 주소
C	막혔다	막힌 칸 번호(S1~S10) + 화면 캡처

오늘의 성공 기준 A다. 터미널의 배포 완료 메시지가 아니라, Table Editor에서 **내 눈으로 내 행을 본 것** - Day9부터 이어지는 원칙이 오늘도 동일하다.

C를 올린 학생은 칸 번호만 있어도 된다 - 원인 설명은 안 붙여도 된다. 칸 번호가 강사에게 가장 빠른 단서다.

무엇을 어디서 가져왔고, 무엇이 오늘 새로 생겼나

항목	이전	Day11 오늘
폼 주제	Day10 과제 "선생님 신청하기" (제출만 되던 테스트 폼)	동일 주제 - 오늘은 저장까지 연결
폼 구조·스타일	Day8 해피팜 대기명단 폼 (과일 주제)	구조·스타일 감각만 계승, 과일 주제는 가져오지 않음
저장소	없음 (Day10 테스트 폼은 값이 남지 않음)	Supabase applications 테이블
서버	없음 / 로컬 Node 서버(Day8 해피팜 기준)	Vercel 서버리스 함수 (api/submit.js)
확인 방법	화면의 "제출됐습니다" 문구뿐	Table Editor 표 확인 + Logs duration_ms 확인(신규)
선택 박스 개념	Day8 카테고리(딸기->strawberry)	지도 과목(코딩 실습->coding) - 화면 글자 vs token 개념 동일 유지
실행/배포	-	npx vercel --prod 배포
실습 도구	없음	Claude Code(Haiku) - 로그인·CLAUDE.md 확인 + 컬럼 추가 + /deploy · /check 단축 명령 + 응답시간 로그 실습
프로젝트 규칙 파일	없음	CLAUDE.md - 절대 규칙 7개 + 단축 명령 4개(/start·/check·/deploy·/fix)

바뀌지 않은 것 "화면의 글자와 서버로 가는 token은 다르다"는 개념, 그리고 POST 요청 -> JSON 응답이라는 골격은 Day8부터 오늘까지 그대로다. 오늘 새로 더해진 것은 그 응답이 표에 영구히 남는다는 것과, 처리 시간을 로그로 켜다는 것 두 가지다.

2026-08-03 빌드 - 폼 UI/API 계약(POST /api/submit, 필드 name·contact·subject·motivation)은 Day11 starter 코드와 1:1 일치. 터미널 실측 원문(Vercel CLI 버전 줄, Production 형식)은 day9-selfstudy-deck(2026-08-02)에서 그대로 재사용했다. Supabase/Vercel 화면은 전부 UI 재현(mock)이다. v3에서 체크리스트 2장과 S10 로그인·CLAUDE.md 확인·/deploy·/check 확장 3장이 새로 추가됐다 - CLAUDE.md 자체는 같은 starter 폴더의 규칙 파일과 1:1 대응한다.